

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

Освітній компонент
«АЛГОРИТМИ І СТРУКТУРИ ДАНИХ»

ЗВІТ
З ПРАКТИЧНОЇ РОБОТИ № 2

Виконав
студент групи КН-25-1
Гордієнко В. Ю.
Перевірив:
доцент кафедри КІЕ
Сидоренко В. М.

Кременчук 2026

Тема: Асимптотична складність алгоритмів. Інші нотації

Мета: набути практичних навичок у розв'язанні задач на оцінку асимптотичної складності алгоритмів у Ω , Θ , o , θ , ω -нотаціях.

Порядок виконання роботи

6. Нехай $f(n) = 6n^3 - 9n^2 + 12n$ і $g(n) = n^3$. Довести, що $f(n) = \Omega(g(n))$.

Розкладемо функцію, щоб виділити n^3 : $f(n) = n^3 + (5n^3 - 9n^2) + 12n$

Знайдемо n_0 :

Подивимось на вираз у дужках: $5n^3 - 9n^2$. Він стає додатним, як тільки $5n^3$ стає більшим за $9n^2$.

Якщо $n = 2$: $5(8) - 9(4) = 40 - 36 = 4$ (підходить, більше 0).

Отже, при $n \geq 2$ вираз у дужках завжди більше нуля.

Оскільки при $n \geq 2$ частини $(5n^3 - 9n^2)$ та $12n$ є додатними, ми можемо їх просто "відкинути", і значення функції від цього тільки зменшиться:

$$6n^3 - 9n^2 + 12n \geq n^3; \quad f(n) \geq n^3$$

Ми знайшли константи $c = 1$ та $n_0 = 2$. Умова $f(n) \geq c * g(n)$ виконується, отже $f(n) = \Omega(g(n))$.

11. Дано функції $f(n) = n^3 + 2n^2 - 5n + 8$, $g(n) = n^4$. Показати, що $f(n) = O(g(n))$, використовуючи метод меж.

Якщо границя $\lim_{n \rightarrow \infty} \left(\frac{n^3}{n^4} + \frac{2n^2}{n^4} - \frac{5n}{n^4} + \frac{8}{n^4} \right) = L$, де L — це скінченне число (включаючи 0), то $f(n) = O(g(n))$.

Спростимо вираз: $\lim_{n \rightarrow \infty} \left(\frac{1}{n} + \frac{2}{n^2} - \frac{5}{n^3} + \frac{8}{n^4} \right)$

Знайдемо значення: Оскільки при $n \rightarrow \infty$ кожен з цих дробів прямує до 0:

$$0 + 0 - 0 + 0 = 0$$

Оскільки границя дорівнює 0, то функція $f(n) = O(g(n))$

Висновки: на цій практичній роботі я набув практичних навичок у розв'язанні задач на оцінку асимптотичної складності алгоритмів у Ω , Θ , o , θ , ω -нотаціях.

Контрольні питання

1. Що таке асимптотична складність алгоритму?

Це оцінка того, як змінюється час роботи або кількість операцій алгоритму, коли розмір вхідних даних n збільшується.

2. Які інші нотації, крім O -нотації, використовуються для вираження асимптотичної складності?

Крім O – нотації, існують: Ω , Θ , o , θ , ω -нотації.

3. Як визначити асимптотичну складність алгоритму за допомогою символів Θ і Ω ?

- $\Omega(g(n))$: Це нижня межа. Алгоритм працює не швидше ніж $g(n)$. Ми шукаємо таку константу c , щоб $f(n) \geq c * g(n)$ для всіх великих n .

- $\Theta(g(n))$: Це точна оцінка. Алгоритм працює точно як $g(n)$ (з точністю до константи). Вона визначається тоді, коли функція є одночасно і $O(g(n))$, і $\Omega(g(n))$.

4. Яка різниця між O -нотацією, Θ -нотацією і Ω -нотацією?

Це можна уявити як математичні порівняння:

- (Big-O): Аналог $\leq$ "Не гірше ніж...". Гарантує, що час не перевищить певну межу (найгірший сценарій).

- Ω (Big-Omega): Аналог $\geq$ "Не краще ніж...". Гарантує мінімальну складність (найкращий сценарій).

- Θ (Big-Theta): Аналог $=$ "Рівно...". Означає, що верхня і нижня межі збігаються.

5. Які основні властивості інших нотацій, таких як o (маленька o), ω (маленька ω) та θ (маленька θ з верхнім індексом)?

Ці нотації використовуються для опису строгих відношень (коли одна функція значно менша або більша за іншу):

- o (o маленьке): Аналог $<$. Означає, що $f(n)$ росте значно повільніше за $g(n)$.

- ω (ω маленька): Аналог $>$. Означає, що $f(n)$ росте значно швидше за $g(n)$.